

BAB 11: IMPLEMENTASI SISTEM AUTENTIKASI PENGGUNA DENGAN CODEIGNITER 4 DAN BOOTSTRAP

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa diharapkan mampu:

1. Merancang dan membuat skema database yang sesuai untuk menyimpan data pengguna dengan menerapkan prinsip keamanan, khususnya dalam penyimpanan kredensial.
2. Membangun Model dalam CodeIgniter 4 yang berfungsi sebagai lapisan abstraksi untuk interaksi dengan tabel pengguna di database.
3. Mengembangkan Controller yang mengelola logika bisnis terkait proses pendaftaran, autentikasi, dan terminasi sesi pengguna.
4. Menerapkan aturan validasi input yang komprehensif pada sisi server untuk menjaga integritas data dan keamanan aplikasi.
5. Merancang antarmuka pengguna (view) yang responsif dan interaktif untuk form pendaftaran dan login dengan memanfaatkan framework Bootstrap.
6. Mengelola state pengguna (sesi) untuk mempertahankan status autentikasi di berbagai halaman aplikasi.
7. Memahami dan menerapkan teknik hashing password yang aman menggunakan fungsi bawaan PHP.
8. Mengintegrasikan mekanisme perlindungan Cross-Site Request Forgery (CSRF) yang disediakan oleh CodeIgniter 4 untuk mengamankan form submission.

Materi Singkat

Autentikasi pengguna merupakan salah satu pilar fundamental dalam pengembangan aplikasi web modern yang memiliki sifat interaktif dan personal. Sistem ini berfungsi sebagai gerbang utama yang memverifikasi identitas individu yang mencoba mengakses suatu sumber daya atau layanan dalam aplikasi. Implementasi autentikasi yang robust tidak hanya membatasi akses berdasarkan peran, tetapi juga melindungi data sensitif pengguna dan memastikan bahwa interaksi dalam aplikasi terjadi secara terkendali. Pada intinya, proses autentikasi melibatkan verifikasi kredensial yang disajikan oleh pengguna—biasanya berupa kombinasi pengenalan unik (seperti alamat email atau nama pengguna) dan sebuah rahasia (password)—terhadap data yang telah disimpan secara aman dalam database.

Dalam konteks pengembangan web, keamanan password adalah aspek yang paling kritis. Penyimpanan password dalam bentuk teks biasa (plaintext) adalah praktik yang sangat berbahaya dan tidak dapat ditoleransi, karena pelanggaran keamanan database dapat langsung mengakibatkan tersebarnya informasi pribadi pengguna secara massal. Untuk mengatasi ini, digunakan teknik *hashing*, sebuah fungsi kriptografis satu arah yang mengubah password menjadi string karakter acak dengan panjang tetap. Fungsi seperti `password_hash()` di PHP menggunakan algoritma modern seperti bcrypt, yang secara inheren dirancang untuk lambat dan *salted* (penambahan data acak unik untuk setiap password), membuatnya sangat resisten

terhadap serangan *rainbow table* dan *brute-force*. CodeIgniter 4, sebagai framework PHP yang mengikuti pola arsitektur Model-View-Controller (MVC), menyediakan struktur yang ideal untuk memisahkan logika bisnis, presentasi, dan akses data, sehingga mempermudah pembangunan sistem autentikasi yang terorganisir, mudah dikelola, dan aman. Model akan menangani komunikasi dengan database `users`, Controller akan mengatur alur logika untuk registrasi, login, dan logout, sementara View akan bertanggung jawab untuk merender form-form yang dibutuhkan pengguna. Framework seperti Bootstrap digunakan untuk mempercepat pengembangan antarmuka pengguna yang responsif dan estetik tanpa perlu menulis banyak kode CSS dari awal.

Bab ini membangun fondasi autentikasi untuk sebuah studi kasus yang lebih luas: **Aplikasi Komplain Pelanggan**. Dalam aplikasi tersebut, fitur login dan registrasi adalah prasyarat utama sebelum pelanggan dapat membuat tiket pengaduan, melampirkan bukti, dan memantau status penyelesaian masalah mereka. Oleh karena itu, database dan struktur tabel yang dibuat dalam bab ini akan dirancang untuk mendukung pengembangan aplikasi tersebut di bab-bab selanjutnya.

Langkah-langkah Praktikum

Praktikum ini akan memandu pembuatan sistem autentikasi dasar yang terdiri dari fungsionalitas pendaftaran pengguna baru, login untuk pengguna terdaftar, dan logout. Diasumsikan lingkungan pengembangan CodeIgniter 4 telah siap dan konsep dasar MVC telah dipahami.

Langkah 1: Persiapan Database dan Struktur Tabel

Langkah awal adalah menyiapkan infrastruktur data yang akan menyimpan informasi pengguna. Ini melibatkan pembuatan database dan tabel dengan struktur yang dirancang untuk keamanan dan efisiensi, serta penamaan yang sesuai dengan studi kasus aplikasi komplain pelanggan.

1. Pembuatan Database:

Buat sebuah database baru pada server MySQL dengan nama yang mencerminkan proyek, misalnya `db_komplain_pelanggan`.

2. Pembuatan Tabel `users`:

Eksekusi query SQL berikut di dalam database `db_komplain_pelanggan` untuk membuat tabel `users`. Tabel ini tidak hanya berfungsi untuk autentikasi, tetapi juga akan menjadi referensi utama untuk data pelanggan dalam aplikasi komplain.

```
CREATE TABLE `users` (  
  `id` INT(11) UNSIGNED NOT NULL AUTO_INCREMENT,  
  `username` VARCHAR(50) NOT NULL,  
  `email` VARCHAR(100) NOT NULL,  
  `password_hash` VARCHAR(255) NOT NULL,  
  `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
```

```

CURRENT_TIMESTAMP,
PRIMARY KEY (`id`),
UNIQUE KEY `uniq_email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Penjelasan Struktur Tabel:

- `id` : Primary key bertipe integer yang akan bertambah secara otomatis (auto-increment). Akan menjadi ID unik setiap pelanggan.
- `username` : Kolom untuk menyimpan nama pengguna (pelanggan).
- `email` : Kolom untuk menyimpan alamat email pelanggan, yang diberi constraint `UNIQUE` untuk memastikan setiap email hanya terdaftar sekali.
- `password_hash` : Kolom terpenting untuk keamanan. Akan menyimpan hasil hashing dari password pelanggan, bukan password aslinya.
- `created_at` , `updated_at` : Kolom timestamp yang secara otomatis mencatat waktu pembuatan dan pembaruan record.

3. Konfigurasi Koneksi Database CodeIgniter 4:

Buka file `app/Config/Database.php` dan sesuaikan parameter koneksi pada bagian `public $default` dengan detail koneksi ke database `db_komplain_pelanggan`.

```

// app/Config/Database.php

public $default = [
    'DSN'          => '',
    'hostname'     => 'localhost',
    'username'     => 'root',          // Sesuaikan dengan username database
    'password'     => '',             // Sesuaikan dengan password database
    'database'     => 'db_komplain_pelanggan', // Nama database untuk aplikasi
    'driver'       => 'MySQLi',
    'dbPrefix'     => '',
    'pConnect'     => false,
    'DBDebug'      => (ENVIRONMENT !== 'production'),
    'charset'      => 'utf8mb4',
    'DBCollat'     => 'utf8mb4_unicode_ci',
    'swapPre'      => '',
    'encrypt'      => false,
    'compress'     => false,
    'strictOn'     => false,
];

```

```
'failover' => [],  
'port'     => 3306,  
];
```

Langkah 2: Pembuatan Model untuk Interaksi Data

Model berfungsi sebagai perantara antara aplikasi dan tabel `users`. Model ini akan menangani semua operasi database terkait pengguna (pelanggan), seperti mengambil data berdasarkan email atau menyimpan data pengguna baru.

Buat file baru bernama `UserModel.php` di dalam direktori `app/Models/` dan tambahkan kode berikut:

```
<?php namespace App\Models;  
  
use CodeIgniter\Model;  
  
class UserModel extends Model  
{  
    protected $table           = 'users';  
    protected $primaryKey      = 'id';  
    protected $useAutoIncrement = true;  
    protected $returnType      = 'array'; // Mengembalikan hasil sebagai array  
    protected $useSoftDeletes   = false;  
    protected $protectFields    = true;  
    protected $allowedFields    = ['username', 'email', 'password_hash'];  
  
    // Mengaktifkan fitur timestamp otomatis  
    protected $useTimestamps = true;  
    protected $dateFormat    = 'datetime';  
    protected $createdField   = 'created_at';  
    protected $updatedField   = 'updated_at';  
  
    /**  
     * Mengambil data pengguna berdasarkan alamat email.  
     *  
     * @param string $email Alamat email yang akan dicari.  
     * @return array|null Data pengguna jika ditemukan, null jika tidak.  
     */  
    public function findByEmail(string $email): ?array  
    {  
        return $this->where('email', $email)->first();  
    }  
}
```

```
}  
}
```

Penjelasan Kode:

- `$table` : Menentukan nama tabel yang akan digunakan oleh model ini.
- `$allowedFields` : Daftar kolom yang diizinkan untuk diisi secara massal. Ini adalah fitur keamanan penting untuk mencegah *Mass Assignment*.
- `$useTimestamps` : Jika diatur `true`, CodeIgniter akan secara otomatis mengisi kolom `created_at` dan `updated_at`.
- `findByEmail()` : Metode kustom untuk mencari satu baris data pengguna berdasarkan kolom `email`.

Langkah 3: Pembuatan Controller untuk Logika Autentikasi

Controller adalah otak dari sistem autentikasi. Controller ini akan berisi logika untuk menampilkan form, memvalidasi input, berinteraksi dengan model, dan mengelola sesi pengguna (pelanggan).

Buat file baru bernama `Auth.php` di dalam direktori `app/Controllers/` dan tambahkan kode berikut:

```
<?php namespace App\Controllers;  
  
use App\Models\UserModel;  
  
class Auth extends BaseController  
{  
    /**  
     * Menampilkan halaman pendaftaran dan memproses data pendaftaran.  
     */  
    public function register()  
    {  
        return view('auth/register_view');  
    }  
  
    /**  
     * Menampilkan halaman login dan memproses data login.  
     */  
    public function login()  
    {
```

```

        return view('auth/login_view');
    }

    /**
     * Menghancurkan sesi dan mengalihkan pengguna ke halaman login.
     */
    public function logout()
    {
        session()->destroy();
        return redirect()->to('/auth/login')->with('success', 'Anda telah keluar
dari sistem.');
```

Penjelasan Kode:

- **Metode `register()`** :
 - Memeriksa apakah request datang dari metode POST.
 - Mendefinisikan aturan validasi, termasuk `is_unique[users.email]`.
 - Jika validasi berhasil, password di-hash menggunakan `password_hash()`.
 - `session()->setFlashdata()` digunakan untuk menyimpan pesan sukses.
- **Metode `login()`** :
 - Memvalidasi input email dan password.
 - Mengambil data pengguna dari database menggunakan `UserModel`.
 - Fungsi `password_verify()` digunakan untuk memverifikasi password.
 - Jika kredensial valid, data pengguna disimpan dalam sesi.
 - Pengguna diarahkan ke `/pelanggan/dashboard` sebagai contoh halaman setelah login.
- **Metode `logout()`** :
 - `session()->destroy()` menghapus semua data sesi.

Langkah 4: Pembuatan View untuk Antarmuka Pengguna

View adalah bagian yang bertanggung jawab atas tampilan yang dilihat oleh pengguna. Kita akan membuat tiga view: pendaftaran, login, dan halaman dashboard pelanggan.

1. Persiapan Direktori dan Bootstrap:

- Buat direktori baru bernama `auth` di dalam `app/Views/`.
- Sertakan CSS Bootstrap di dalam setiap view.

2. View Pendaftaran (`app/Views/auth/register_view.php`)

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pendaftaran Akun Pelanggan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body>
  <div class="container mt-5" style="max-width: 500px;">
    <h2 class="text-center mb-4">Buat Akun Pelanggan Baru</h2>

    <form action="<?= site_url('/auth/register') ?>" method="post">
      <?= csrf_field() ?>
      <div class="mb-3">
        <label for="username" class="form-label">Nama
Lengkap</label>
        <input type="text" class="form-control" id="username"
name="username" required>
      </div>
      <div class="mb-3">
        <label for="email" class="form-label">Email</label>
        <input type="email" class="form-control" id="email"
name="email" required>
      </div>
      <div class="mb-3">
        <label for="password" class="form-label">Password</label>
        <input type="password" class="form-control" id="password"
name="password" required>
      </div>
      <div class="mb-3">
        <label for="confirm_password" class="form-label">Konfirmasi
Password</label>
        <input type="password" class="form-control"
id="confirm_password" name="confirm_password" required>
      </div>
      <button type="submit" class="btn btn-primary w-
100">Daftar</button>
    </form>
    <p class="text-center mt-3">Sudah punya akun? <a href="<?=

```

```

    site_url('/auth/login') ?>">Masuk di sini</a></p>
  </div>
</body>
</html>

```

3. View Login (`app/Views/auth/login_view.php`)

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Masuk ke Akun Pelanggan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body>
  <div class="container mt-5" style="max-width: 500px;">
    <h2 class="text-center mb-4">Masuk Akun Pelanggan</h2>

    <form action="<?= site_url('/auth/login') ?>" method="post">
      <?= csrf_field() ?>
      <div class="mb-3">
        <label for="email" class="form-label">Email</label>
        <input type="email" class="form-control" id="email"
name="email" required>
      </div>
      <div class="mb-3">
        <label for="password" class="form-label">Password</label>
        <input type="password" class="form-control" id="password"
name="password" required>
      </div>
      <button type="submit" class="btn btn-primary w-
100">Masuk</button>
    </form>
    <p class="text-center mt-3">Belum punya akun? <a href="<?=
site_url('/auth/register') ?>">Daftar di sini</a></p>
  </div>
</body>
</html>

```

4. View Dashboard Pelanggan (app/Views/pelanggan/dashboard_view.php)

Buat direktori pelanggan di app/Views/. Kemudian buat file dashboard_view.php di dalamnya.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard Pelanggan</title>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-light bg-light">
        <div class="container-fluid">
            <a class="navbar-brand" href="#">Sistem Komplain Pelanggan</a>
            <div class="d-flex">
                <span class="navbar-text me-3">
                    Selamat datang, <strong><?= session()->get('username') ?
></strong>!
                </span>
                <a class="btn btn-outline-danger" href="<?=
site_url('/auth/logout') ?>" role="button">Keluar</a>
            </div>
        </div>
    </nav>
    <div class="container mt-4">
        <h3>Dashboard Pelanggan</h3>
        <p>Selamat datang di halaman utama pelanggan. Dari sini, Anda dapat
mengelola pengaduan Anda.</p>
        <?php if (session()->getFlashdata('success')): ?>
            <div class="alert alert-info"><?= session()-
>getFlashdata('success') ?></div>
        <?php endif; ?>
        <!-- Konten dashboard pelanggan akan ditambahkan di bab berikutnya -
->
    </div>
```

```
</body>
</html>
```

Penjelasan Kode View:

- `site_url()` : Fungsi helper CodeIgniter untuk menghasilkan URL absolut.
- `csrf_field()` : Menghasilkan input field tersembunyi yang berisi token CSRF [24].
- `set_value()` : Fungsi helper form untuk mengisi kembali nilai input jika validasi gagal.
- `session()->getFlashdata()` : Menampilkan pesan yang telah diset pada controller.
- `session()->get()` : Mengambil data yang disimpan dalam sesi, seperti `username` .

Langkah 5: Konfigurasi Routing

Routing memetakan URL ke controller dan metode tertentu. Buka file `app/Config/Routes.php` dan tambahkan rute berikut:

```
// app/Config/Routes.php

$routes->get('/', 'Home::index'); // Rute default

// Grup rute untuk autentikasi
$routes->group('auth', ['namespace' => 'App\Controllers'], function($routes) {
    $routes->match(['get', 'post'], 'register', 'Auth::register');
    $routes->match(['get', 'post'], 'login', 'Auth::login');
    $routes->get('logout', 'Auth::logout');
});

// Rute untuk dashboard pelanggan, dilindungi oleh filter di BaseController
$routes->get('pelanggan/dashboard', 'Pelanggan::dashboard');
```

Penjelasan Kode:

- `$routes->group()` : Mengelompokkan beberapa rute dengan awalan yang sama (`auth`).
- `$routes->match()` : Membuat rute yang merespons baik metode GET maupun POST.
- Rute untuk `pelanggan/dashboard` diasumsikan akan menuju ke controller `Pelanggan` .

Langkah 6: Membuat Controller Pelanggan dan Filter Login

Agar halaman dashboard hanya dapat diakses oleh pengguna yang sudah login, diperlukan mekanisme proteksi.

1. Buat Controller `Pelanggan.php` di `app/Controllers/` :

```
<?php namespace App\Controllers;

class Pelanggan extends BaseController
{
    public function dashboard()
    {
        // Cek apakah pengguna sudah login
        if (!session()->get('isLoggedIn')) {
            return redirect()->to('/auth/login');
        }

        $data['title'] = 'Dashboard Pelanggan';
        return view('pelanggan/dashboard_view', $data);
    }
}
```

2. Pengujian Sistem:

- Jalankan server pengembangan CodeIgniter 4 (`php spark serve`).
- Akses `http://localhost:8080/auth/register` untuk mendaftarkan pengguna baru.
- Setelah pendaftaran, coba login melalui `http://localhost:8080/auth/login` .
- Jika berhasil, sistem akan mengalihkan ke `http://localhost:8080/pelanggan/dashboard` .
- Coba akses `http://localhost:8080/pelanggan/dashboard` langsung dari browser tanpa login. Pengguna seharusnya diarahkan kembali ke halaman login.

Latihan

1. **Implementasi Pesan Validasi Kustom:** Ubah pesan validasi default CodeIgniter 4 menjadi Bahasa Indonesia.
2. **Penambahan Fitur "Remember Me":** Tambahkan checkbox "Ingat Saya" pada form login. Jika dicentang, sesi pengguna harus memiliki durasi yang lebih lama.
3. **Penggunaan Template Layout:** Jika modul sebelumnya membahas tentang template layout, refactor ketiga file view untuk menggunakan template tersebut.
4. **Halaman Profil Pelanggan:** Buat halaman profil (`/pelanggan/profil`) yang menampilkan informasi pelanggan yang sedang login dan memungkinkan pengguna untuk memperbarui username atau email.

Referensi

[11] Validation — CodeIgniter 4.6.3 documentation.

<https://codeigniter4.github.io/userguide/libraries/validation.html>

[20] Security — CodeIgniter 4.6.3 documentation. https://codeigniter.com/user_guide/libraries/security.html

[24] Create News Items — CodeIgniter 4.6.3 documentation.

https://codeigniter4.github.io/userguide/tutorial/create_news_items.html

[30] Session Library — CodeIgniter 4.6.3 documentation.

https://codeigniter.com/user_guide/libraries/sessions.html